

Production Planning Optimization Problem

Alejandro Treny Ortega

Mixed Integer Linear Programming (MILP) with Gurobi

Author: Alejandro Treny Ortega

Date: November 2025

Topic: Production optimization with piecewise linear profits and resource constraints

Problem Overview

A company manufactures three discrete products, each with:

- Production capacity limit: 60 units maximum
- Fixed startup costs
- Declining marginal profit rates (piecewise linear)
- Resource consumption across 4 different resource types

Objective: Maximize total profit (gross profit - fixed costs)

Constraints: Resource availability, capacity limits, logical dependencies

1. Installation & Setup

Install Gurobi and required packages:

```
Gurobi imported successfully
```

2. Define Sets and Parameters

Data Input Section

Problem Parameters:

Products: [1, 2, 3]

Resources: [1, 2, 3, 4]

Fixed costs: {1: 40, 2: 50, 3: 45}

Resource availability: {1: 1500, 2: 1900, 3: 1800, 4: 1200}

Profit segments (product_id -> [(profit/unit, capacity), ...]):

Product 1: [(4, 10), (3, 50)]

Product 2: [(6, 8), (4, 52)]

Product 3: [(5, 10), (2.5, 10), (1, 40)]

Resource consumption per unit (columns: products 1, 2, 3):

Resource	1	2	3
1	12	15	10
2	15	14	12
3	11	13	9
4	13	12	15

3. Model Creation & Variable Definition

Restricted license - for non-production use only - expires 2027-11-29

Model created with variables:

- Binary variables (y): 3 (product production decisions)
- Integer variables (x): 3 (total quantities)
- Integer variables (v): 7 (segment quantities)
- Binary variables (w): 4 (segment activation)

Total variables: 17

4. Constraint Definitions and Mathematical Formulation

Let $J = \{1, 2, 3\}$ denote products, $I = \{1, 2, 3, 4\}$ resources and $K_j = \{1, \dots, m_j\}$ the ordered profit segments for product j . The parameter dictionaries above specify fixed costs F_j , resource capacities b_i , consumption a_{ij} , segment profits p_{jk} and segment capacities c_{jk} . Mathematical segment indices start at one; Python indices start at zero.

Variable	Meaning	Domain
x_j	Total production of product j	Nonnegative integer
y_j	Whether product j is produced	Binary
v_{jk}	Units assigned to segment k	Nonnegative integer
$w_{jk}, k \geq 2$	Whether a later segment is activated	Binary

The objective subtracts fixed charges from gross contribution:

$$\max \sum_{j \in J} \sum_{k \in K_j} p_{jk} v_{jk} - \sum_{j \in J} F_j y_j.$$

Production aggregates across segments and respects resource capacities:

$$x_j = \sum_{k \in K_j} v_{jk} \quad (j \in J), \quad \sum_{j \in J} a_{ij} x_j \leq b_i \quad (i \in I).$$

Activation means strictly positive integer production, bounded by 60 units. The implication from product 3 to product 1 is enforced separately:

$$y_j \leq x_j \leq 60 y_j \quad (j \in J), \quad y_3 \leq y_1.$$

Segment quantities obey their capacities and require an active product:

$$v_{jk} \leq c_{jk}, \quad v_{jk} \leq c_{jk} y_j \quad (j \in J, k \in K_j).$$

Later segments can be used only after filling the preceding segment:

$$v_{j,k-1} \geq c_{j,k-1} w_{jk}, \quad v_{jk} \leq c_{jk} w_{jk}, \quad w_{jk} \leq y_j \quad (j \in J, k = 2, \dots, m_j).$$

Together these constraints prevent skipped profit tiers and ensure that the fixed-cost indicators match actual production. The implemented formulation has 17 variables (10 integer quantities and 7 binary indicators) and 40 constraints, including three minimum-production constraints. Some segment capacity constraints are redundant but retained for correspondence with the course formulation.

```
All constraints added successfully
Total constraints: 40
```

5. Objective Function

Maximize: Total Profit = Gross Profit - Fixed Costs

```
Objective function set: Maximize (Gross Profit - Fixed Costs)
```

6. Solver Configuration

Note: These parameters disable presolve, heuristics and cuts to inspect the resulting branch-and-bound search

```
Set parameter Presolve to value 0
Set parameter Heuristics to value 0
Set parameter Cuts to value 0
Solver parameters configured:
- Presolve: DISABLED (for academic analysis)
- Heuristics: DISABLED
- Cuts: DISABLED
```

7. Solve the Model

This cell runs the Gurobi optimizer:

```
=====
STARTING OPTIMIZATION
=====
Gurobi Optimizer version 13.0.2 build v13.0.2rc1 (mac64[arm] - Darwin 25.6.0 25G83)

CPU model: Apple M5 Pro
Thread count: 18 physical cores, 18 logical processors, using up to 18 threads

Non-default parameters:
Heuristics 0
Cuts 0
Presolve 0

Optimize a model with 40 rows, 17 columns and 81 nonzeros (Max)
Model fingerprint: 0xdd8fdf56
Model has 10 linear objective coefficients
Variable types: 0 continuous, 17 integer (7 binary)
Coefficient statistics:
  Matrix range      [1e+00, 6e+01]
  Objective range   [1e+00, 5e+01]
  Bounds range      [1e+00, 1e+00]
  RHS range         [8e+00, 2e+03]

Variable types: 0 continuous, 17 integer (7 binary)
```

Root relaxation: objective 2.983077e+02, 8 iterations, 0.00 seconds (0.00 work units)

Nodes		Current Node			Objective Bounds			Work	
Expl	Unexpl	Obj	Depth	IntInf	Incumbent	BestBd	Gap	It/Node	Time
	0	0	298.30769	0	5	- 298.30769	-	-	0s
	0	0	286.76923	0	5	- 286.76923	-	-	0s
	0	2	286.76923	0	5	- 286.76923	-	-	0s
*	3	6		2	239.0000000	286.71389	20.0%	3.3	0s
*	13	9		4	284.0000000	285.33333	0.47%	2.0	0s

Explored 18 nodes (38 simplex iterations) in 0.00 seconds (0.00 work units)
Thread count was 18 (of 18 available processors)

Solution count 2: 284 239

Optimal solution found (tolerance 1.00e-04)
Best objective 2.840000000000e+02, best bound 2.840000000000e+02, gap 0.0000%

=====

OPTIMIZATION COMPLETE

=====

8. Solution Report

=====

OPTIMAL SOLUTION SUMMARY

=====

Optimal net profit: €284.00

PRODUCT-LEVEL DECISIONS

Product 1:
Produce? (y_1): 1
Total quantity (x_1): 36 units
Fixed cost: €40

Product 2:
Produce? (y_2): 1
Total quantity (x_2): 60 units
Fixed cost: €50

Product 3:
Produce? (y_3): 0
Total quantity (x_3): 0 units
Fixed cost: Not incurred (not produced)

SEGMENT-LEVEL PRODUCTION BREAKDOWN

Product 1 segments:
Segment 1: 10/10 units @ €4/unit = €40

Segment 2: 26/50 units @ €3/unit = €78

Product 2 segments:

Segment 1: 8/8 units @ €6/unit = €48

Segment 2: 52/52 units @ €4/unit = €208

Product 3 segments:

Segment 1: 0/10 units (not used)

Segment 2: 0/10 units (not used)

Segment 3: 0/40 units (not used)

RESOURCE UTILIZATION

Resource 1: 1332/1500 units (88.8%) AVAILABLE

Resource 2: 1380/1900 units (72.6%) AVAILABLE

Resource 3: 1176/1800 units (65.3%) AVAILABLE

Resource 4: 1188/1200 units (99.0%) HIGH UTILIZATION

FINANCIAL SUMMARY

Gross profit: € 374.00

Fixed costs: € 90.00

Net profit: € 284.00

SOLVER STATISTICS

Branch-and-Bound nodes explored: 18.0

Optimality proven: YES

9. Advanced Analysis: Solution Insights

MANAGERIAL INSIGHTS

1 PRODUCT PROFITABILITY ANALYSIS

Product 1:

Gross contribution: €118.00

Fixed costs: €40

Contribution: €78.00

Net / fixed cost: 195%

Product 2:

Gross contribution: €256.00

Fixed costs: €50

Contribution: €206.00

Net / fixed cost: 412%

Product 3: NOT PRODUCED

2 RESOURCE UTILIZATION ANALYSIS

Most utilized resource: Resource 4
Utilization: 99.0%
Available slack: 12 units

High utilization alone does not establish a profitable capacity expansion.

3 WHAT-IF ANALYSIS

+1 units of resource 4: net profit EUR 287.00, gain EUR 3.00 before expansion costs
+12 units of resource 4: net profit EUR 287.00, gain EUR 3.00 before expansion costs
+13 units of resource 4: net profit EUR 287.00, gain EUR 3.00 before expansion costs
+100 units of resource 4: net profit EUR 308.00, gain EUR 24.00 before expansion costs

10. Sensitivity & What-If Analysis

Explore how the solution changes with different parameters:

=====

SENSITIVITY ANALYSIS TEMPLATE

=====

To perform sensitivity analysis, modify parameters and re-run:

Example 1: Increase Product 2 profit in segment 1 from 6 to 7 EUR/unit

```
# Modify the segments dictionary:
segments[2] = [(7, 8), (4, 52)] # Changed 6 -> 7
```

```
# Then re-run cells starting from 'Model Creation & Variable Definition'
```

Example 2: Increase availability of most-utilized resource (Resource 4)

```
# Modify resource availability:
b[4] = 1300 # Increased from 1200
```

```
# Then rebuild the model, constraints and objective, and re-run the optimization
```

Example 3: Remove the logical constraint (if Product 3 is made, Product 1 must be made)

```
# Comment out this line in the Constraint Definition section:
# model.addConstr(y[3] <= y[1], name="R4_Logic_P3_P1")
```

```
# Then rebuild the model, constraints and objective, and re-run the optimization
```

11. Export Results to CSV

Product-Level Results:

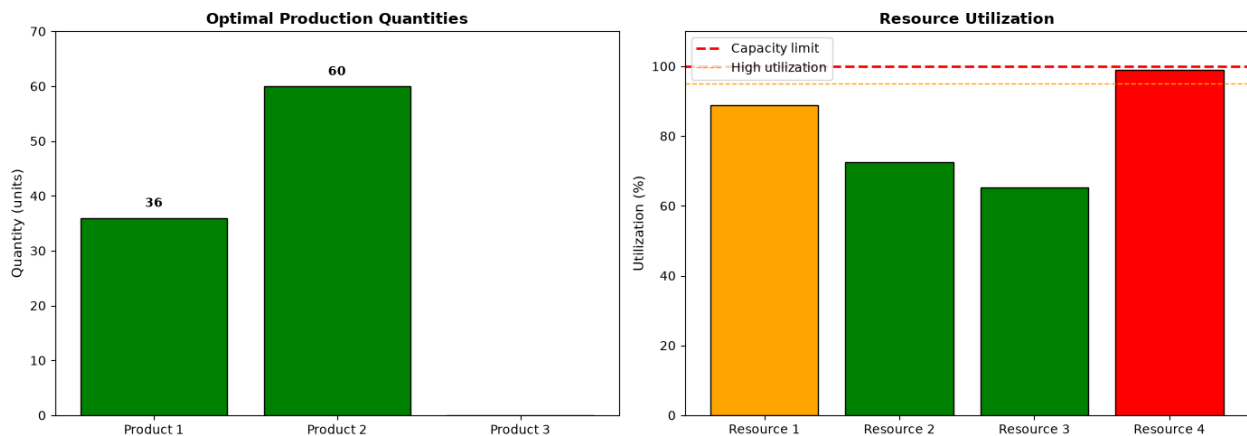
	Product	Produce	Quantity	Fixed_Cost	Gross_Profit
0	1	1	36.0	40	118.0
1	2	1	60.0	50	256.0
2	3	0	0.0	0	0.0

Resource Utilization:

	Resource	Used	Available	Utilization_%	Slack
0	1	1332.0	1500	88.800000	168.0
1	2	1380.0	1900	72.631579	520.0
2	3	1176.0	1800	65.333333	624.0
3	4	1188.0	1200	99.000000	12.0

12. Visualization (Optional)

Create visualizations of the solution:



Visualization saved as 'optimization_results.png'

Summary

This notebook implements a complete **Mixed Integer Linear Programming (MILP)** solution using Gurobi to solve a production planning optimization problem.

Key Results:

- Optimal profit identified
- Resource utilization and integer capacity scenarios computed
- Sensitivity analysis framework provided
- Exportable results for reporting

Next Steps:

1. Modify parameters for what-if analysis
2. Compare re-optimized integer capacity scenarios
3. Test different business scenarios
4. Use results for strategic planning